

Compte Rendu : Installation d'une Plateforme de Développement IoT (ESP32)

1. Objectifs du TP

L'objectif de cette séance était de mettre en place un environnement de développement complet pour l'Internet des Objets (IoT) sous GNU/Linux et de valider cette installation par la programmation d'une carte **NodeMCU ESP32**.

2. Matériels et Logiciels utilisés

- **Matériel** : Carte NodeMCU ESP32 (SoC Tensilica Xtensa LX6).
- **Éditeur** : Visual Studio Code.
- **Écosystème** : Extension PlatformIO (Framework Arduino).
- **Système d'exploitation** : GNU/Linux (Debian/Ubuntu).

3. Déroulement des Activités

Activité 1 : Installation de l'environnement

Nous avons installé **Visual Studio Code** ainsi que les dépendances système nécessaires (python3-venv et python3-virtualenv).

Dans VS Code, nous avons ajouté les extensions suivantes :

- **C/C++ (Microsoft)** : Pour la coloration syntaxique et le débogage.
- **PlatformIO IDE** : Pour la gestion de projet, la compilation croisée et le téléversement.

Activité 2 : Configuration des accès série

Sous Linux, l'accès aux ports série nécessite des permissions spécifiques. Nous avons :

1. Téléchargé et installé les règles **udev** de PlatformIO (99-platformio-udev.rules) dans `/etc/udev/rules.d/`.
2. Ajouté notre utilisateur aux groupes `dialout` et `plugdev` pour autoriser la communication avec la carte via USB.

Activité 3 : Identification du matériel

En analysant le schéma structurel de la carte (Figure 4 du document), nous avons identifié que la **LED bleue** intégrée est connectée au **GPIO 2** de l'ESP32.

4. Réalisation Logicielle (Code Final)

Plutôt qu'un simple clignotement, nous avons développé un programme plus complexe permettant d'émettre un signal de secours **SOS en code Morse**.

Logique du programme :

- Un point (.) dure 250 ms.
- Un trait (-) dure 3 fois la durée d'un point (750 ms).
- Le message "SOS" est répété toutes les 2 secondes.

Code source (main.cpp)

```
#include <Arduino.h>

const int8_t LED = GPIO_NUM_2; // LED bleue identifiée sur le GPIO 2
const int TEMPO = 250;

// --- Fonctions de base ---
void point() {
    digitalWrite(LED, HIGH); delay(TEMPO); // LED allumée
    digitalWrite(LED, LOW); delay(TEMPO); // Espace entre symboles
}

void tiret() {
    digitalWrite(LED, HIGH); delay(3 * TEMPO); // LED allumée 3x plus
longtemps
    digitalWrite(LED, LOW); delay(TEMPO); // Espace entre symboles
}

// --- Traduction d'un caractère ---
void caractere(char lettre) {
    switch (lettre) {
        case 'S': case 's':
            point(); point(); point();
            break;
        case 'O': case 'o':
            tiret(); tiret(); tiret();
            break;
        case ' ':
            delay(4 * TEMPO);
            return;
        default:
            return;
    }
}
```

```

    }
    delay(2 * TEMPO); // Pause entre chaque lettre
}

// --- Envoi d'une chaîne complète ---
void morse(String texte) {
    for (int i = 0; i < texte.length(); i++) {
        caractere(texte[i]);
    }
}

void setup() {
    pinMode(LED, OUTPUT); // Configuration du GPIO 2 en sortie
}

void loop() {
    morse("SOS"); // Envoi du message SOS
    delay(2000);  // Temporisation avant répétition
}

```

5. Conclusion

L'installation de la plateforme de développement est opérationnelle. La compilation et le téléversement via PlatformIO s'effectuent sans erreur. La carte ESP32 exécute correctement le code Morse, ce qui valide à la fois la chaîne de compilation et la communication série entre le PC et le microcontrôleur.